

Beginner — Scenarios 1–8

1. Student Grade Calculator

Create an array of student objects:

```
[
  { name: "Arun", marks: 85 },
  { name: "Priya", marks: 72 },
  { name: "Kumar", marks: 45 }
]
```

Requirements:

- Calculate grades using conditions.
- Loop through all students.
- Create a function `calculateGrade(marks)`.
- Display each student's name, marks, and grade.

Practice: Objects + Arrays + Loops + Conditions + Functions

2. Employee Salary Calculator

Create employee objects:

```
{
  name: "John",
  salary: 50000,
  experience: 6
}
```

Calculate:

- Experience < 3 → 5% bonus
- Experience 3–5 → 10%
- Experience > 5 → 15%

Create a function:

`calculateSalary(employee)`

3. Product Price Calculator

Create an array of products:

```
[  
  { name: "Laptop", price: 60000 },  
  { name: "Mouse", price: 1500 },  
  { name: "Keyboard", price: 3000 }  
]
```

Requirements:

- Calculate total price.
 - Apply discount:
 - Total > ₹50,000 → 20%
 - Total > ₹20,000 → 10%
 - Otherwise → no discount.
 - Display final amount.
-

4. Find Largest Number

Given:

[25, 10, 75, 42, 90, 18]

Create a function:

findLargest(numbers)

Do **not** use `Math.max()`.

Use:

- Array
 - Loop
 - Condition
 - Function
-

5. Find Even and Odd Numbers

Given:

[10, 23, 44, 51, 67, 82, 90]

Create a function that separates numbers into:

```
{
  even: [],
  odd: []
}
```

Example:

```
{
  even: [10, 44, 82, 90],
  odd: [23, 51, 67]
}
```

6. Shopping Cart Total

Create:

```
const cart = [
  { name: "Laptop", price: 60000, quantity: 1 },
  { name: "Mouse", price: 1500, quantity: 2 },
  { name: "Keyboard", price: 3000, quantity: 1 }
];
```

Calculate:

item total = price × quantity
cart total

Then apply discount based on total.

7. Login Validation

Create a user object:

```
{
  username: "admin",
  password: "admin123"
}
```

Create:

login(username, password)

Conditions:

Correct username + password → Login successful

Correct username + wrong password → Invalid password

Wrong username → User not found

8. Temperature Converter

Create functions:

celsiusToFahrenheit()

fahrenheitToCelsius()

Use conditions to determine whether the temperature is:

Very Cold

Cold

Normal

Hot

Very Hot

Intermediate — Scenarios 9–17

9. Employee Search System

Create an employee array:

```
[  
  { id: 101, name: "Arun", department: "IT" },  
  { id: 102, name: "Priya", department: "HR" },  
  { id: 103, name: "John", department: "IT" }  
]
```

Create functions:

findEmployeeById(id)

findEmployeesByDepartment(department)

Return matching employee objects.

10. Inventory Management

Create:

```
[  
  { name: "Laptop", stock: 10 },  
  { name: "Mouse", stock: 0 },  
  { name: "Keyboard", stock: 5 }  
]
```

Create functions:

```
checkStock()  
getOutOfStockProducts()  
getLowStockProducts()
```

Rules:

```
stock === 0 → Out of Stock  
stock <= 5 → Low Stock  
stock > 5 → Available
```

11. Bank Account Transaction System

Create:

```
const account = {  
  name: "Murali",  
  balance: 50000,  
  transactions: []  
};
```

Create functions:

```
deposit()  
withdraw()  
getBalance()
```

Rules:

- Deposit must be positive.
- Withdrawal cannot exceed balance.
- Store every transaction in `transactions`.

Example transaction:

```
{  
  type: "deposit",  
  amount: 5000  
}
```

12. Expense Tracker

Create:

```
[  
  { category: "Food", amount: 500 },  
  { category: "Travel", amount: 1000 },  
  { category: "Food", amount: 700 },  
  { category: "Shopping", amount: 2000 }  
]
```

Create functions:

```
calculateTotal()  
getFoodExpenses()  
getHighestExpense()
```

Also calculate:

```
Food total  
Travel total  
Shopping total  
Overall total
```

13. Employee Attendance System

Create:

```
[  
  { name: "Arun", presentDays: 22, totalDays: 25 },  
  { name: "Priya", presentDays: 18, totalDays: 25 }  
]
```

Calculate attendance percentage.

Rules:

>= 90% → Excellent
>= 75% → Good
>= 60% → Average
< 60% → Poor

Create:

calculateAttendance(employee)

14. Product Search & Filtering

Given:

```
[  
  { name: "iPhone", category: "Mobile", price: 80000 },  
  { name: "Samsung", category: "Mobile", price: 60000 },  
  { name: "Dell", category: "Laptop", price: 70000 },  
  { name: "HP", category: "Laptop", price: 50000 }  
]
```

Create functions:

```
findByCategory()  
findByPriceRange()  
findProductByName()
```

Example:

findByPriceRange(50000, 70000)

15. Order Status System

Create orders:

```
[  
  { id: 1, customer: "Arun", status: "shipped" },  
  { id: 2, customer: "Priya", status: "pending" },  
  { id: 3, customer: "John", status: "delivered" }  
]
```

Create a function:

getOrderMessage(order)

Return:

pending → Order is being processed
shipped → Order is on the way
delivered → Order delivered successfully
cancelled → Order cancelled

16. Shopping Cart Validation

Given:

```
[  
  { name: "Laptop", price: 60000, quantity: 1 },  
  { name: "Mouse", price: 1500, quantity: 0 },  
  { name: "Keyboard", price: 3000, quantity: 2 }  
]
```

Create functions:

```
validateCart()  
calculateCartTotal()  
getInvalidItems()
```

Rules:

- Quantity must be greater than 0.
 - Price must be greater than 0.
 - Product name cannot be empty.
-

17. Student Performance Analyzer

Create:

```
[  
  {  
    name: "Arun",  
    marks: [80, 75, 90, 85]  
  },  
  {  
    name: "Priya",  
    marks: [60, 70, 65, 72]  
  }  
]
```


Create functions:

```
calculateAverage()  
findHighestMark()  
findLowestMark()  
calculateGrade()
```

Generate:

```
{  
  name: "Arun",  
  average: 82.5,  
  highest: 90,  
  lowest: 75,  
  grade: "A"  
}
```

Advanced — Scenarios 18–25

18. E-Commerce Order Processing

Create an order:

```
{  
  customer: {  
    name: "Murali",  
    city: "Bangalore"  
  },  
  items: [  
    { name: "Laptop", price: 60000, quantity: 1 },  
    { name: "Mouse", price: 1500, quantity: 2 }  
  ],  
  paymentMethod: "UPI"  
}
```

Create functions:

```
calculateSubtotal()  
calculateDiscount()  
calculateTax()  
calculateFinalAmount()  
validateOrder()
```

Final output:

```
{
  subtotal: 63000,
  discount: 6300,
  tax: 10206,
  finalAmount: 66906
}
```

19. Employee Department Analyzer

Given 20+ employees:

```
[
  {
    name: "Arun",
    department: "IT",
    salary: 80000,
    experience: 6
  },
  ...
]
```

Create functions to determine:

Highest-paid employee

Lowest-paid employee

Average salary

Employees by department

Employees with > 5 years experience

Employees earning > ₹100,000

Avoid built-in methods initially. Use loops and conditions.

20. Banking Transaction Analyzer

Given:

```
[
  { type: "credit", amount: 5000 },
  { type: "debit", amount: 2000 },
  { type: "credit", amount: 10000 },
]
```

```
{ type: "debit", amount: 3000 }  
]
```

Create:

```
calculateBalance()  
getTotalCredits()  
getTotalDebits()  
getLargestTransaction()
```

Output:

```
{  
  totalCredits: 15000,  
  totalDebits: 5000,  
  balance: 10000  
}
```

21. Duplicate Data Detector

Given:

```
[  
  { id: 101, email: "a@test.com" },  
  { id: 102, email: "b@test.com" },  
  { id: 103, email: "a@test.com" }  
]
```

Create:

```
findDuplicateEmails(users)
```

Return:

```
["a@test.com"]
```

Then extend it to detect duplicate IDs.

22. Inventory Reorder System

Given:

```
[
```

```
{
  name: "Laptop",
  stock: 3,
  reorderLevel: 5,
  reorderQuantity: 10
},
{
  name: "Mouse",
  stock: 20,
  reorderLevel: 5,
  reorderQuantity: 10
}
]
```

Create:

generateReorderList(products)

If:

stock <= reorderLevel

add the product to the reorder list.

Expected:

```
[
  {
    name: "Laptop",
    orderQuantity: 10
  }
]
```

23. Multi-Level Employee Salary Analysis

Create:

```
[
  {
    name: "Arun",
    department: "IT",
    salary: 100000,
    performance: {
      rating: 5
    }
  }
]
```

```
},  
{  
  name: "Priya",  
  department: "HR",  
  salary: 80000,  
  performance: {  
    rating: 3  
  }  
}  
}
```

Bonus rules:

Rating 5 → 20%
Rating 4 → 15%
Rating 3 → 10%
Rating < 3 → 5%

Create:

```
calculateBonus(employee)  
calculateFinalSalary(employee)
```

Then find the employee receiving the highest bonus.

24. Complete Shopping Cart Engine

Build a mini shopping-cart engine using only the five topics.

Your data:

```
const cart = [  
  {  
    product: {  
      id: 1,  
      name: "Laptop",  
      price: 60000  
    },  
    quantity: 1  
  },  
  {  
    product: {  
      id: 2,  
      name: "Mouse",  

```

```
    price: 1500
  },
  quantity: 2
}
];
```

Implement:

```
addProduct()
removeProduct()
updateQuantity()
calculateSubtotal()
calculateDiscount()
calculateTax()
calculateFinalTotal()
```

Add business rules for:

Discount
Tax
Invalid quantity
Out-of-stock products
Minimum order amount

This is an excellent interview-style exercise.

25. Mini Employee Management System ★

Build a complete console-based employee management system.

Employee:

```
{
  id: 101,
  name: "Arun",
  department: "IT",
  salary: 90000,
  experience: 6
}
```

Implement:

```
addEmployee()
removeEmployee()
```

findEmployee()
updateEmployee()
getEmployeesByDepartment()
getHighestSalaryEmployee()
getLowestSalaryEmployee()
calculateAverageSalary()
getExperiencedEmployees()
calculateBonus()

Business rules:

Experience $\geq 10 \rightarrow 20\%$ bonus
Experience $\geq 5 \rightarrow 15\%$ bonus
Experience $\geq 3 \rightarrow 10\%$ bonus
Experience $< 3 \rightarrow 5\%$ bonus

Then generate an employee report:

Employee Report

Total Employees: 10
Average Salary: ₹85,000
Highest Salary: ₹1,50,000
Lowest Salary: ₹35,000
IT Employees: 4
HR Employees: 3
Experienced Employees: 6
Total Bonus: ₹1,25,000
